

Hackathon Prerequisites & Resources Guide

To work through the hackathon challenges, you'll need two things ready beforehand:

- **A coding setup** — for example, Claude Code, OpenCode, Codex, Cursor, or any other coding agent you're comfortable using.
- **Model / compute access** — either through hosted APIs, free model providers, cloud GPU runtimes, local hardware, or any other setup that works for you.

You're free to use any subscriptions, credits, tools, or infrastructure you already have. If you don't have a setup yet, this guide includes a few free options to get started.

The examples outlined are not mandatory setups — they're simply practical starting points in case you don't have your own setup.

Please have your coding environment and compute/model access set up and tested before the hackathon so you can start building right away.

Coding Setup Guide

If you already use Claude Code, OpenCode, Codex, Cursor, or another coding agent with your own model access or subscription, you are free to continue using that setup.

If you do not already have a setup, there are many workable combinations: a coding agent paired with a free model provider or gateway such as OpenRouter, another provider with a free tier, or a local model. You are encouraged to explore these options and pick what works best for you.

Example setup: The walkthrough below is just one possible setup. It uses Claude Code as the coding agent and OpenRouter to access a free model. You do not need to use this exact combination. You can instead use:

- A different coding agent with OpenRouter
- A different model provider (orcarouter, omnirouter, etc) that offers free access
- A local model running on your own machine
- Any other setup you are comfortable with and have tested beforehand

For the example below, have these ready before you begin:

- Claude Code installed on your machine
- An OpenRouter account
- A terminal you are comfortable using

Example: Claude Code + OpenRouter + a Free Model

Claude Code is simply the coding agent in this example. OpenRouter provides access to the model that Claude Code will use. The same general idea can be applied to other compatible coding agents and model providers.

1. Create an OpenRouter account and API key

Go to openrouter.ai and create an account. For this example, you can use models that are available for free, so you do not need to add credits.

Once you are logged in:

1. Go to Home.
2. Select API Keys from the left panel.
3. Create a new API key.
4. Copy the generated API key.

It should look something like:

```
sk-or-v1-...
```

Keep it private: Treat the API key like a password. Do not share it or commit it to your codebase.

2. Choose a free model

For this example, we will use:

```
nvidia/nemotron-3.5-lightning:free
```

You can browse currently available free text models here: [OpenRouter - Free Models](#).

Free-model availability can change. Pick a model you want to try and copy its model ID. You can always switch models later if one is rate-limited or does not work well with your coding agent.

3. Point Claude Code to OpenRouter

Run the following commands in the same terminal session where you plan to use Claude Code.

macOS / Linux / WSL

```
export ANTHROPIC_BASE_URL="https://openrouter.ai/api"
export ANTHROPIC_AUTH_TOKEN "<your-api-key>"
export ANTHROPIC_MODEL="nvidia/nemotron-3.5-lightning:free"
```

Replace <your-api-key> with the API key you copied from OpenRouter. If you selected another model, replace the value of ANTHROPIC_MODEL with that model ID.

Windows PowerShell

```
$env:ANTHROPIC_BASE_URL="https://openrouter.ai/api"  
$env:ANTHROPIC_AUTH_TOKEN="<your-api-key>"  
$env:ANTHROPIC_MODEL="nvidia/nemotron-3.5-lightning:free"
```

Note: These environment variables apply to the current terminal session. If you close the terminal and open a new one, you may need to set them again.

4. Verify that the setup works

Run a tiny one-off request:

```
claude -p "Reply with exactly: OK"
```

If the setup is working, you should receive:

```
OK
```

This confirms that Claude Code can make a request through the model you configured.

5. Confirm the selected model

Start Claude Code:

```
claude
```

Inside Claude Code, run:

```
/model
```

Confirm that the model you intended to use is selected. You can also run:

```
/status
```

Use this to confirm that Claude Code is routing through OpenRouter with the base URL and authentication token you configured.

Test Your Setup Before the Hackathon

Do not stop at the OK test. Try a few small coding tasks and make sure the setup is usable for you.

Free models can sometimes be rate-limited, temporarily unavailable, slower under demand, or more/less compatible with a particular coding agent. If one model is not working reliably:

- Open the free-model catalogue.
- Pick another free model.
- Update your selected model.
- Run the verification again.
- Try a small coding task.

Recommendation: have one or two backup model IDs or an alternative setup ready before the event.

Note

Different tools have different configuration steps, so if you choose another combination, follow that tool's documentation and test the complete flow beforehand. For reference, OpenRouter also documents integrations for [Claude Code](#) and [Codex CLI](#).

Part 1 — Free GPU Runtimes

Use these to run your own models. Both give you a T4 GPU in the browser, with no setup on your own machine.

Option 1 — Kaggle Notebooks

1. Create an account at kaggle.com
2. Go to **Settings** → **Phone Verification** and verify your number (*the GPU option stays locked until you do this*)
3. Click **Create** → **Notebook**
4. In the right sidebar, open **Session options** → **Accelerator** and choose **GPU T4 x2**
5. Run this in the first cell to confirm it worked:

```
import torch
print(torch.cuda.is_available())  # should print True
```

You get 30 GPU hours per week, and the remaining amount is shown in the sidebar.

Option 2 — Google Colab

1. Open colab.research.google.com and sign in with a Google account
2. Click **New notebook**
3. Go to **Runtime** → **Change runtime type**, select **T4 GPU**, and click **Save**
4. Run this in the first cell to confirm it worked:

Two things to keep in mind

Check the GPU is actually on. If the code above prints **False**, you're on a CPU and everything will run far slower. Recheck the accelerator setting.

Save your work as you go. Sessions can disconnect. Save results and checkpoints to [/kaggle/working/](#) on Kaggle, or to Google Drive on Colab:

```
from google.colab import drive
drive.mount('/content/drive')
```

Option 3 — Lightning AI

1. Open [Lightning AI](#) and create a free account
2. You get **5 free credits when you sign up**
3. Add a **credit/debit card** to your account to unlock **25 additional free credits**, giving you up to **\$30 of free credits**
4. Create a new **Lightning Studio**
5. Start with the free CPU instance for installing packages and preparing your dataset
6. When you're ready to fine-tune, switch the Studio to a GPU such as **T4, L4, L40S, or A100**
7. Run this to confirm the GPU is available:

Two things to keep in mind

Add your card to unlock the additional credits.

You can create an account without a credit card, but adding a card unlocks the additional free credits that are useful for GPU fine-tuning.

Turn the GPU off when you're not using it.

GPU instances consume credits while running. Install dependencies, prepare the dataset, and write/debug your code on the free CPU instance first. Switch to the GPU only when you're ready to train or fine-tune.

Lightning Studios have persistent storage, so your files remain available when the Studio stops. Still, save model checkpoints regularly during fine-tuning.

Option 4 — Modal

Serverless GPUs. You write a normal Python script, mark a function with a decorator, and Modal runs it on a cloud GPU and bills per second.

Set up the account from the dashboard:

1. Go to modal.com/signup. Signing up with an existing **GitHub** account is the standard route

2. Open **Settings** → [Usage & Billing](#) and click **Add Payment Method**. This opens a Stripe-hosted page where you add your **credit/debit card**
3. Modal's docs state plainly that **you must have a payment method on file in order to use Modal**, so do this before the event rather than on the day
4. The free **Starter** plan then gives you **\$30 of compute credit per month**. Check the balance and current usage on the same **Usage & Billing** page

Then connect your machine:

6. Install the CLI and authenticate — this opens a browser window and writes an API token to your machine:

```
pip install modal #use "pipx install modal" If using mac
modal setup
```

7. Save this as `gpu_test.py` to confirm GPU access works:

```
import modal

app = modal.App("gpu-test")

image = modal.Image.debian_slim().pip_install("torch", "numpy")

@app.function(gpu="T4", image=image)
def check_gpu():
    import torch

    return torch.cuda.is_available()

@app.local_entrypoint()
def main():
    print(check_gpu.remote())
```

8. Run it:

```
modal run gpu_test.py
```

In the dashboard afterwards: your jobs appear under **Apps**, with logs for each run. **Secrets** are the right place to store API keys instead of hardcoding them, **Volumes** hold anything that must survive between runs, and **Budgets** (under Settings) let you cap workspace spend — worth setting if you're nervous about the card on file.

You can follow this guide: <https://modal.com/docs/guide>

Three things to keep in mind

A card is required, but the free credits still cover you. Modal requires a payment method on file to use the platform at all. The \$30 monthly credit is deducted from usage before you're charged, so normal hackathon usage should not reach a bill — but the card is live, so set a Budget, watch the Usage & Billing page, and stop long jobs when you're done.

Credits reset monthly, they don't roll over. Use them within the month; unused credits won't carry forward to the hackathon if you set the account up too early.

Part 2 — Hosted Model APIs

Use these when you want to call a large model without running it yourself. No GPU needed.

Option 1 — AI Grants India x Flytbase

You will get a link to this form on Hackathon Day !

Steps :

1. Fill the form
2. At the end of the form you'll be redirected to a WhatsApp Number, text on that number
3. In a few minutes you'll receive your API key for OpenAI gpt-5.6-luna !

Note:- *This API key is highly rate limited (around 4 RPM) so use it considering this constraint*

Option 2 — NVIDIA NIM

A catalog of 100+ hosted models, including vision models. Free, no credit card.

1. Go to build.nvidia.com and create an account

2. Verify your phone number when prompted — API access stays locked until you do
3. Browse the vision models at build.nvidia.com/explore/vision
4. Open any model page and click **Get API Key**. Copy the key immediately — you usually only see it once.
5. Call it using the OpenAI SDK, just with a different base URL:

```
from openai import OpenAI

client = OpenAI(
    base_url="https://integrate.api.nvidia.com/v1",
    api_key="nvapi-YOUR-KEY-HERE"
)

response = client.chat.completions.create(
    model="meta/llama-3.2-90b-vision-instruct",
    messages=[{"role": "user", "content": "Describe this scene."}]
)
print(response.choices[0].message.content)
```

Good to know: the free tier allows around 40 requests per minute. Any tool or library that already speaks the OpenAI API format will work here by changing the base URL and model name.

Option 3 — Gemini API (free tier)

Useful because Gemini accepts video and images directly. Free, no credit card.

1. Go to aistudio.google.com and sign in with a Google account
2. Click **Get API key** → **Create API key**
3. Install the SDK:

```
pip install google-genai
```

4. Make your first call:

```
from google import genai

client = genai.Client(api_key="YOUR-KEY-HERE")
```

```
response = client.models.generate_content(  
    model="gemini-2.5-flash",  
    contents="Say hello."  
)  
print(response.text)
```

Good to know:

- The free tier covers **Flash and Flash-Lite models only**. Pro models now require billing, so build with Flash from the start rather than prototyping on Pro and hitting a wall.
 - **Limits are per project, not per key.** Creating extra API keys inside the same project will not give you more quota.
 - Your exact limits are shown in AI Studio. Check there rather than relying on numbers from blog posts, since Google changes them without notice.
 - If you get a **429** error, you've hit a rate limit. Add a short wait and retry rather than hammering the endpoint.
-

Before you arrive

- ☐ Kaggle account created **and phone-verified**
- ☐ Colab opened at least once
- ☐ Setup Modal account
- ☐ NVIDIA API key generated and saved
- ☐ Gemini API key generated and saved
- ☐ One test call made successfully with each